

Atlas — 한국어의역본

이 문서는 arXiv:2505.23735 「Atlas: Learning to Optimally Memorize the Context at Test Time」 (Behrouz et al. (Google Research))의 한국어 의역본입니다. 스터디용이며 공식 번역이아닙니다. 수식 · 수치 · 그림은 원문 기준이고, 그림은 원저자의 것입니다. 자연스러운 이해를 위해 문장은의역했습니다.

저자: Ali Behrouz, Zeman Li, Praneeth Kacham, Majid Daliri, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, Vahab Mirrokni (Google) **arXiv:** 2505.23735v1 [cs.CL], 2025년 5월 29일

초록 (Abstract)

Transformer는 in-context 검색 (retrieval)에서의 뛰어난 성능과 대규모 학습능력 덕분에 시퀀스 모델링의 대표적인 백본으로 자리 잡았다. 그러나 attention의 이차 (quadratic) 메모리 · 시간 복잡도는 긴시퀀스에 대한 적용을 제약했고, 이는 modern recurrent neural network(즉, long-term recurrent memory module)와 같은 효율적인 대안 구조를탐구하도록 만들었다. 이러한 최근 recurrent 모델들은 다양한 downstream 과제에서 성공을 거두었지만, 여전히 **long context 이해와더 긴 시퀀스로의 외삽 (extrapolation)**에서는 어려움을 겪는다.

우리는 이 한계가 세 가지 서로 독립적인 설계 결함에서 비롯된다고 본다: (1) 메모리 구조와 입력의 feature mapping에 의해 제약되는 **한정된메모리 용량 (capacity)**, (2) 메모리를 오직 마지막 입력에 대해서만최적화하는 업데이트의 **online 한 성질**, (3) 고정 크기 메모리를다루는 **덜 표현력 있는 메모리 관리 (management)**. 이 세 측면을모두 개선하기 위해 우리는 **Atlas**를 제시한다. Atlas는 높은 용량을가는 long-term memory module로, 현재와 과거 토큰을 함께 고려하여메모리를 최적화함으로써 **문맥 (context)을 기억하는 법을 학습**하며, long-term memory 모델의 online 한 성질을 극복한다.

이 통찰을 바탕으로 우리는 원래 Transformer 구조를 **엄밀하게일반화 (strict generalization)**하는 새로운 Transformer 류 구조군 **DeepTransformers**를 제시한다. 언어 모델링, 상식추론 (common-sense reasoning), recall-intensive 과제, long-context 이해과제에 대한 실험 결과, Atlas는 Transformer와 최근의 선형 recurrent 모델들의 성능을 능가한다. 또한 Atlas는 Titans의 long context 성능을 더욱개선하여, BABILong 벤치마크의 **10M 문맥 길이에서 +80% 정확도**를달성한다.

1 서론 (Introduction)

Attention 모듈 (Bahdanau et al. 2014)은 현대 딥러닝 구조의 핵심 구성요소로, 확장성과 in-context 검색 성능에서 두각을 나타낸다. 원리적으로 attention은 associative memory(연상 기억)로 기능한다. 즉 토큰 쌍 사이의직접적인 의존성을 계산하여 key-value 매핑을 저장하고, query-key 유사도를통해 이를 검색한다. 그러나 이 쌍별 의존성 계산은 정확하지만**이차의 공간 · 시간 복잡도**를 유발하여, long context 이해 · 기억 · 모델링에서의 적용을 제약한다.

이 한계를 극복하기 위한 최근 연구들은 더 효율적이면서도 효과적인 recurrent neural network를 설계해 왔다. 이러한 modern recurrent 구조들은 **‘attentional bias’**라 불리는 내부 목적함수 (internal objective)를 최적화하는 associative memory 모듈로 통일적으로 이해할 수있다 (Behrouz, Razaviyayn, et al. 2025). Transformer의 커져 가는 KV cache와 달리, 이 모델들은 고정 크기 메모리를 사용하므로 더 나은 메모리관리가 필수적이다. 이에 따라 RNN 메모리 관리를 개선하려는 세 방향의노력이 있었다: (i) **학습 규칙 (learning rule)** — additive learning에서 DeltaNet의 Delta rule까지; (ii) **Forget(Retention) gate** — RetNet의 입력 비의존적 게이

팅에서 Titans·RWKV-7 의적응적 게이팅까지; (iii) **메모리 구조** — 벡터형 메모리에서 신경망기반 deep memory 모듈까지.

이런 개선에도 불구하고 이 모델들은 여전히 long context 이해, in-context 검색, 더 긴 시퀀스로의 외삽에서 고전한다. 우리는 이 결함이 세 가지설계적 측면에서 비롯된다고 본다:

1. **메모리 업데이트의 online 한 성질** — 과거 메모리 상태는 유지한채 현재 토큰에 대해서만 메모리를 최적화하여, 더 넓은 문맥을 고려하지 못하고 개별 토큰만을 기억한다.
2. **메모리의 한정된 용량** — 구조와 key-value feature mapping 이완벽하게 매핑 가능한 key-value 쌍의 수를 제약한다.
3. **메모리 관리 (내부 목적함수의 optimizer) 의 표현력 부족** — 대부분의 최근 모델이 gradient descent 를 사용하는데, 이는 토큰 동역학의 1 차 (first-order) 정보에만 의존하여 메모리가 그릇된 국소최소점 (spurious local minima) 에 수렴하고 덜 효과적인 key-value 매핑을 학습하게 만든다.

메모리 관점 (Memory Perspective)

Associative memory — 서로 다른 개체나 사건을 매핑하는 능력 — 는 인간학습에서 분리할 수 없는 요소이며, 최신 딥러닝 구조를 이 관점으로 이해하려는 연구들을 촉발했다. 이 관점에서 메모리는 입력에 의해 유발되는 신경망의 갱신으로 정의된다. 즉 입력이 더 **놀라울수록 (surprising)** 메모리에 더 큰 영향을 주며 더 잘 기억된다. 따라서 효과적인 “**surprise 지표**” 를 찾는 것이 이러한 메모리 모듈 설계의 핵심이다.

기존 구조들은 거의 모두 현재 입력에 기반해 메모리를 갱신하는 surprise 지표를 사용한다. 그러나 하나의 사건 (토큰들의 시퀀스) 은 오랜 시간 동안 일관되게 놀랍지 않더라도 기억할 만할 수 있다. 이 문제를 해결하기 위해 Titans(Behrouz, Zhong, et al. 2024) 는 surprise 지표를 “**순간적 (momentary)**” 놀라움과 “**과거 (past)**” 놀라움의 두부분으로 나누어, 현재 입력으로 메모리를 갱신할 때 과거 입력의 누적놀라움을 함께 반영하도록 제안했다. 그러나 이 설계 역시 개별 토큰을 기억하다 보면 문맥을 놓칠 수 있다. 그래서 본 연구에서는 **국소적 (또는 전역적) 문맥 창 (window) 의 놀라움을 측정**하는 long-term neural memory 모듈을 제시한다. 즉 test time 에 (토큰) 문맥을 기억하는 법을 학습한다.

이 논문 전반에서 우리는 “**Test Time Memorization (테스트 시점기억)**” 이라는 용어를 사용한다. 이 과정은 정보를 오직 전역 문맥 안에서만 저장·검색할 뿐, 모델의 사전학습된 핵심 파라미터 (즉 outer-loop) 나 초기 상태를 갱신하지 않기 때문이다. 메모리가 한 번 비워지면 어떤 지속적 학습이나 기술 습득도 새로운 독립적 문맥으로 넘어가지 않는다. 따라서 우리는 “test time training” 보다 “test time memorization” 이라는 표현을 선호한다.

기여 (Contributions)

앞서 언급한 세 한계 — (1) online 한 성질, (2) 한정된 메모리 용량, (3) 표현력 부족한 메모리 관리 — 를 극복하기 위해, 우리는 높은 용량을 가지며 토큰이 아닌 **문맥을 기억**할 수 있는 long-term neural memory 모듈을 설계한다. 구체적으로:

- **메모리 용량과 그 병목에 대한 더 나은 이해.** 한정된 메모리 용량을 개선하기 위해 입력 토큰에 대한 고차 (higher-order) feature mapping (예: 다항식 커널) 을 제안한다. 왜 더 깊은 메모리 모듈과/또는 고차 feature mapping 이 메모리 용량 — 메모리가 완벽하게 매핑할 수 있는 선형 독립 key-value 연상의 최대 개수 — 을 높이는지 이론적으로 정당화한다.
- **새로운 표현력 있는 학습 규칙.** 최근 recurrent 모델들의 online 한 성질을 극복하기 위해, 마지막 토큰뿐 아니라 주어진 문맥 창안의 모든 과거 토큰에 기반해 메모리를 갱신하는 sliding window 업데이트 규칙, 즉 **Omega rule** 을 제시한다.
- **Transformer 의 엄밀한 일반화.** Omega rule 이 전역 및 국소 (즉 Sliding Window Attention, SWA) softmax attention 과 연결됨을 보이고, Transformer 를 엄밀하게 일반화하는 새 Transformer 류 구조인 **DeepTransformers** 와 그 sliding window 변형 **SWDT** 를 제시한다. 또한 deep memory 의 역할을 보이기 위한 새로운 baseline **Deep Linear Attention (DLA)** 를 제시한다.

- **더 나은 메모리 관리를 갖춘 새 메모리 모듈.** key와 query에다항식 feature를 쓰면서 Omega rule과 gradient descent로 메모리를 갱신하는 **OmegaNet**을 제시한다. 나아가 메모리 관리를 더 강화하기 위해, 내부 메모리 갱신에 **Muon optimizer**(Jordan et al. 2024)를 사용하는 **Atlas**를 도입한다. OmegaNet과 Atlas 모두 병렬화 가능한 학습 알고리즘을 활용하여, online 버전(즉 문맥 창 = 1) 대비 큰 오버헤드 없이 빠르게 학습된다. 우리가 아는 한 Atlas는 (근사적) **2차(second-order) 정보**로 메모리를 최적화하는 최초의 병렬화 가능한 recurrent 구조다(즉 locally optimal 메모리 모듈을 갖는다).
- **다양한 downstream 과제에서의 개선.** 언어 모델링, 상식 추론, recall-intensive, needle-in-haystack 과제에서 DeepTransformers·OmegaNet·Atlas는 modern 선형 RNN, 국소 attention(SWA), Transformer를 능가한다.

증명, 추가 실험, 관련 연구, 실험 세부는 부록에 있다.

2 예비 지식 (Preliminaries)

표기 (Notations). $x \in \mathbb{R}^{N \times d_{in}}$ 을 입력, M_t 를 시각 t 에서 메모리 M 의 상태, K, V, Q 를 각각 key·value·query 행렬로 둔다. 시각 t 에 해당하는 벡터는 k_t, v_t, q_t 로 쓴다. Behrouz, Razaviyayn, et al. (2025)를 따라 $\ell(M_t; k_t, v_t)$ 를 **attentional bias**(내부 메모리 목적함수)로 표기한다. 메모리 모듈 $M(\cdot)$ 의 구조로는 $L_M \geq 1$ 개 층과 residual connection을 갖는 단순 MLP를 쓰지만, 모든 정식화는 다른 메모리 구조(예: $L_M = 1$ 인 선형 행렬형 메모리)에도 손쉽게 적용된다. 필요할 때 메모리 모듈을 $\theta_M := \{W_1, \dots, W_{L_M}, \dots\}$ 로 매개변수화한다.

2.1 배경 (Backgrounds)

Attention. Attention은 Transformer의 associative memory로 작동하는 핵심 구성 요소다. 입력 $x \in \mathbb{R}^{N \times d_{in}}$ 에 대해, 입력 의존적 $Q = xW_Q$, $K = xW_K$, $V = xW_V$ 로부터 causal attention은 출력 $y \in \mathbb{R}^{N \times d_{in}}$ 를 다음과 같이 계산한다:

$$y_i = \frac{1}{Z_i} \sum_{j=1}^i \exp\left(q_i^\top k_j / \sqrt{d_{in}}\right) v_j, \quad Z_i = \sum_{\ell=1}^i \exp\left(q_i^\top k_\ell / \sqrt{d_{in}}\right). \quad (1)$$

이 식은 각 출력이 과거 value들의 query-key 유사도 가중 평균임을 뜻한다. 단순·병렬화 가능한 학습과 recall-intensive 과제에서의 효과에도 불구하고, attention은 토큰당 출력을 계산하는데 최소 $N \times d$ 연산이 필요하여 생성 과정과 long-context 확장에서 큰 약점을 갖는다. 대안 구조들을 두 부류로 나눠 살펴본다: (1) 선형·얇은(shallow) 메모리 recurrent 모델, (2) deep memory 모듈.

(선형) Recurrent 모델. 선형 RNN은 병렬화 가능한 선형 시간 학습과 준수한 성능으로 효율적인 Transformer 대안으로 주목받았다. Hebbian 또는 Delta 학습 규칙에 기반한 초기 modern RNN들은 데이터를 벡터형 또는 행렬형 메모리로 압축한다. $M_t \in \mathbb{R}^{d \times n}$ 을 메모리($n = 1$ 이면 벡터형), $k, v \in \mathbb{R}^d$ 를 key·value로 둘 때 일반적 정식화는 다음과 같다:

$$M_t = A_t * M_{t-1} + v_t k_t^\top, \quad (2)$$

여기서 $*$ 는 임의의 associative 연산자, A_t 는 데이터(비) 의존적 대각 또는 저랭크+항등 행렬이다. 이 식은 새 key-value 외적을 이전 상태(감쇠된)에 더해 넣는 선형 갱신을 뜻한다. 효율적이지만 이 메모리는 문맥이 길어질수록 넘칠 수 있고, forget gate가 관리에 도움을 주더라도 표현력은 선형 구조에 갇혀 있다.

Deep memory 모듈. 메모리의 표현력 한계를 극복하고 유효 문맥 길이를 늘리기 위해, 최근 연구들은 deep memory 모듈을 갖는 구조에 주목한다. 이들은 메모리가 gradient descent(모멘텀

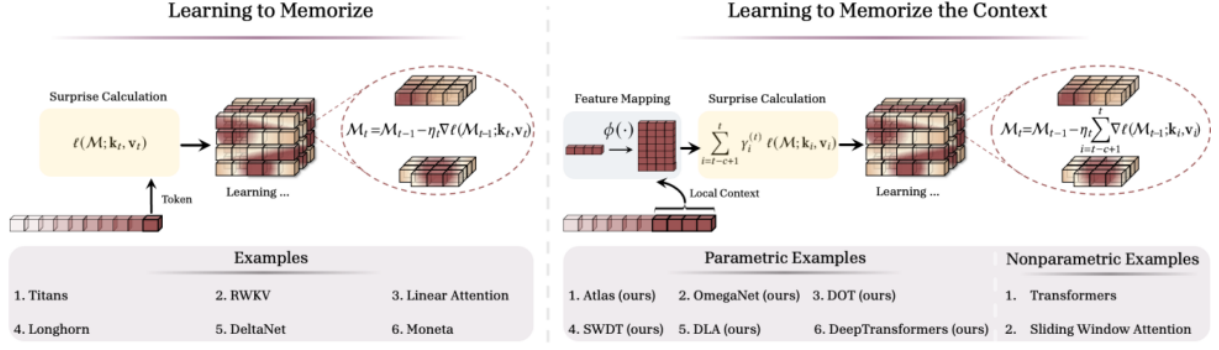


그림 1: (왼쪽) 개별 토큰을 기억하는 학습과 (오른쪽) 문맥을 기억하는 학습의 비교. 출처: 원저자, arXiv:2505.23735 Fig.1

포함)로 갱신되는 deep MLP 인 메타러닝 관점에 기반한다. Behrouz, Razaviyayn, et al. (2025)는 인기 있는 시퀀스 모델들을 test time memorization의 사례로 통일하는 프레임워크를 제시했다. 즉 시퀀스 모델은 attentional bias라는 내부 메모리 목적 함수를 최적화하여 주어진 key와 value 사이의 근본 매핑을 학습하는 associative memory 모듈이다. Associative memory는 형식적으로 다음과 같이 정의된다:

정의 1 (Behrouz, Razaviyayn, et al. 2025). key 집합 $\mathcal{K} \subseteq \mathbb{R}^{d_k}$ 와 value 집합 $\mathcal{V} \subseteq \mathbb{R}^{d_v}$ 가 주어질 때, associative memory는 매핑 $M: \mathcal{K} \rightarrow \mathcal{V}$ 이다. 이 매핑의 학습은 메모리의 유형과 우선순위를 결정하는 목적 함수 $\mathcal{L}$ (Attentional Bias)에 기반한다:

$$M^* = \arg \min_M \mathcal{L}(M(\mathcal{K}); \mathcal{V}). \quad (3)$$

이 목적 함수를 반복적 알고리즘 (예: gradient descent)으로 최적화하면 메모리 갱신 규칙이 나온다. 따라서 시퀀스 모델은 두 최적화 수준을 갖는 메타 in-context learner다:

1. **Inner Loop:** 메모리 모듈의 파라미터 $\theta_M = \{W_1, W_2, \dots, W_{L_M}, \dots\}$ 가 최적화되는 곳. 이때 다른 모든 파라미터는 하이퍼파라미터로 취급되어 고정된다.
2. **Outer Loop:** 선형 projection, MLP 층, convolution 등 모델의 나머지 파라미터가 최적화되는 곳.

우리의 용어는 이 프레임워크에 기반하므로, 완전한 recurrent 정식화 대신 모델을 (1) 메모리 구조, (2) 내부 목적 함수 (attentional bias), (3) 메모리 학습 알고리즘 (optimizer)으로 서술한다.

표 1: 최근 modern recurrent neural network 요약

다섯 특성으로 구조들을 비교한다: (1) Dynamic decay(동적 감쇠), (2) Deep neural memory, (3) 비선형 메모리 용량, (4) **Locally optimal:** 토큰에 대한 2차 정보를 (근사적으로) 사용해 메모리를 관리, (5) **Flexible context:** 문맥을 유연하게 기억하는 능력. $\phi(\cdot)$ 와 $\phi^*(\cdot)$ 는 각각 다항식·무한차원 feature mapping이다 (식 22 참조). † 행렬형 메모리 버전 기준. ‡ NP: Nonparametric. * 정규화 (normalization) 없음.

모델	Attentional Bias ℓ	Optimizer	Dyn. Decay	Deep Mem.	비선형 용량 $\dagger$	Locally Opt.	Flexible Ctx.
Attention	$\sum_{t=1}^L \ M_t k_t - \text{NP} \ddagger\ _2^2$		$\times$	$\times$	$\checkmark$	$\checkmark$	$\times$
SWA	$\sum_{t=c}^L \ M_t k_t - \text{NP}\ _2^2$		$\times$	$\times$	$\checkmark$	$\checkmark$	$\checkmark$
Linear	$\langle M_t k_t, v_t \rangle$	GD	$\times$	$\times$	$\times$	$\times$	$\times$
Attention							
RetNet	$\langle M_t k_t, v_t \rangle$	GD	$\times$	$\times$	$\times$	$\times$	$\times$
GLA	$\langle M_t k_t, v_t \rangle$	GD	$\checkmark$	$\times$	$\times$	$\times$	$\times$
PolySketch-For.	$\langle M_t k_t, v_t \rangle^p$	GD	$\times$	$\times$	$\checkmark$	$\times$	$\times$
TTT	$\ M_t(k_t) - v_t\ _2^2$	GD	$\times$	$\checkmark$	$\times$	$\times$	$\times$
DeltaNet	$\ M_t k_t - v_t\ _2^2$	GD	$\times$	$\times$	$\times$	$\times$	$\times$
Longhorn	$\ M_t k_t - v_t\ _2^2$	Implicit GD	$\times$	$\times$	$\times$	$\times$	$\times$
Gated	$\ M_t k_t - v_t\ _2^2$	GD	$\checkmark$	$\times$	$\times$	$\times$	$\times$
DeltaNet	$\ M_t k_t - v_t\ _2^2$						
RWKV-7	$\ M_t k_t - v_t\ _2^2$	GD	$\checkmark$	$\times$	$\times$	$\times$	$\times$
Titans	$\ M_t(k_t) - v_t\ _2^2$	GD w/ M.*	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$
Moneta	$\ M_t(k_t) - v_t\ _p^p$	GD	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$
Memora	$\ M_t(k_t) - v_t\ _2^2$	GD	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$
DLA (ours)	$\langle M_t(\phi(k_t)), v_t \rangle$	GD	$\checkmark$	$\checkmark$	$\times$	$\times$	$\times$
DeepTransform (ours)	$\langle M_t(\phi^*(k_t)), v_t \rangle$	GD	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\times$
SWDT (ours)	$\sum_{i=c}^L \langle M_t(\phi^*(k_i)), v_i \rangle$	GD	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$
OmegaNet (ours)	$\sum_{i=c}^L \gamma_i \ M_t(\phi(k_i)) - v_i\ _2^2$	GD	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$
Dot* (ours)	$\sum_{i=c}^L \gamma_i \ M_t(\phi(k_i)) - v_i\ _2^2$	GD	$\checkmark$	$\checkmark$	$\checkmark$	$\times$	$\checkmark$
Atlas (ours)	$\sum_{i=c}^L \gamma_i \ M_t(\phi(k_i)) - v_i\ _2^2$	Muon	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

Atlas 의 메모리 쓰기 연산은 $M_t = \alpha_t M_{t-1} - \eta_t \text{NS-5}(S_t)$, $S_t = \theta_t S_{t-1} - \nabla \ell(M_{t-1}; k_t, v_t)$ 이다. Atlas 는이 표에서 유일하게 **Locally Optimal** 과 **Flexible Context** 를 동시에 만족하는 병렬화 가능한 recurrent 모델이다.

3 테스트 시점에 문맥을 기억하는 법 배우기 (Learning to Memorize the Context at Test Time)

Long-term associative memory 는 인간 학습에 필수적이며 여러 인공 신경망구조에 영감을 주었다. 많은 모델이 과거 데이터를 압축하기 위해행렬형 · 벡터형 메모리를 쓰지만, 최근 연구들은 과거의 추상을파라미터에 부호화하는 deep 비선형 신경 메모리를 옹호한다. 그러나 long-context 추론/이해를 위해서는 이 long-term neural memory 모듈이 여전히 다음을 필요로 한다: (1) **높**

은 용량 —파라미터에 저장 가능한 최대 (key, value) 쌍 수 (§ 3.1); (2) key-value 의복잡한 매핑을 학습할 강력한 내부 목적함수 (attentional bias)(§ 3.2); (3) 고정 크기 메모리를 위한 강력한 메모리관리 (§ 3.2); (4) 대규모 학습을 위한 효율적 병렬 학습과정 (§ 3.3).

3.1 초선형 용량을 갖는 Associative Memory (Associative Memory with Super Linear Capacity)

고정된 수의 메모리 파라미터로 저장할 수 있는 상관없는 (uncorrelated) (key, value) 쌍의 최대 개수는 얼마인가? 이 질문에답하기 위해 가장 단순한 경우 — 행렬 메모리, attentional bias 로 ℓ_2 회귀 손실 $\ell(M_t; k_t, v_t) = \|M_t(k_t) - v_t\|_2^2$, gradient descent 최적화 — 부터 시작한다:

명제 1 (ℓ_2 Attentional Bias 의 용량). $d_v \times d_k$ 개 파라미터를 갖는 행렬형 메모리 M 이 $\ell(M_t; k_t, v_t) = \|M_t k_t - v_t\|_2^2$ 를 gradient descent 로최적화한다고 하자. M 은 선형 독립인 key 를 갖는 (k_i, v_i) 쌍을최대 $O(d_k)$ 개 저장할 수 있다.

이 명제는 delta 갱신 규칙을 쓰는 행렬형 메모리가 파라미터 수에 비해부 (sub) 선형 용량을 가짐을 뜻한다. 즉 크기 M 인 고정 메모리가저장할 수 있는 독립 패턴의 수는 어떤 $c \in \mathbb{R}^+$ 에 대해 $c \times M$ 보다 엄격히 작다. 최근 모델들은 deep memory 모듈을제안하지만, 이것이 용량을 실제로 높이는지는 불분명했다.

정리 1 (Deep Memory 의 효과). $M(\cdot)$ 을 $L_M \geq 2$ 개 층, 입력 차원 d_k , 은닉 차원 d_h 를 갖는 MLP 라 하자. 그러면 $M(\cdot)$ 은 선형 독립 key 를 갖는 (k_i, v_i) 쌍을 최소 $O(d_k d_v)$ 개, 최대 $O(d_k d_v \sum_{i=1}^{L_M} \min_{j \geq i} \{d_h^{(j)}\} d_h^{(i+1)})$ 개저장할 수 있다.

이 정리는 deep memory 가 표현력뿐 아니라 용량도 높이며 그 이점이 깊이에따라 커짐을 뜻한다. 그러나 상한은 여전히 key·value 차원에대해 부이차 (subquadratic) 이므로, long-term memory 가 초선형용량을 달성할 수 있는지가 여전히 문제로 남는다.

용량을 늘리려면 k_t 의 차원이 관건이지만, 모든 key-value 차원을 단순히 키우면 파라미터 수 (d_{in} 당 $O(d_{in})$) 와 메모리사용량이 크게 늘어난다. 이를 해결하기 위해, key 와 query 에 대해분리 가능한 커널 (separable kernel) $\sigma(x, y) = \phi(x)^\top \phi(y)$ 를 쓸 것을 제안한다. 그 예로 차수 p 이하의 다항식 커널에 집중한다. $p \in \mathbb{N}$ 에 대해 $\phi_p(x) = [x^\beta]_{|\beta| \leq p}$ 를 x 의 차수 p 이하다항식 mapping 이라 하자. 정의 1 의 목적함수 $\mathcal{L}(M(\mathcal{K}); \mathcal{V})$ 를 $\mathcal{L}(M(\phi(\mathcal{K})); \mathcal{V})$ 로 대체한다. 이 다항식 mapping 은 입력 projection 에 추가 파라미터 없이 key 의 유효차원을 키운다.

명제 2 (다항식 Mapping 을 이용한 메모리 용량). $\phi_p(\cdot)$ 을 차수 p 이하 다항식 mapping, M 을 $\ell(M_t; \phi_p(k_t), v_t) = \|M_t \phi_p(k_t) - v_t\|_2^2$ 를 gradient descent 로 최적화하는 행렬형 메모리라 하자. M 은 선형 독립 key 를 갖는 (k_i, v_i) 쌍을 최대 $O(d_k^p)$ 개 저장할 수 있다.

이 명제는 key 에 대한 다항식 커널이 입력 projection 파라미터를 늘리지않고도 다항식만큼 더 많은 용량을 사준을 뜻한다. 다항식 mapping 은 두 관점으로도 정당화된다. 선형 메모리와 두 attentional bias $\ell^{(1)}(M_t; k_t, v_t) = \langle M_t k_t, v_t \rangle$, $\ell^{(2)}(M_t; k_t, v_t) = \|M_t \phi(k_t) - v_t\|_2^2$ 를 예로 든다. 이를 gradient descent 로 풀면 다음 recurrence 가 나온다:

$$\ell^{(1)} : \quad M_t = M_{t-1} + \eta_t v_t \phi(k_t)^\top, \quad (\text{Hebbian Rule})$$

$$\ell^{(2)} : \quad M_t = \left(I - \eta_t \phi(k_t) \phi(k_t)^\top \right) M_{t-1} + \eta_t v_t \phi(k_t)^\top. \quad (\text{Delta Rule})$$

앞의 식은 새 연상을 단순히 더하는 Hebbian 규칙, 뒤의 식은 기존 예측을빼고 갱신하는 Delta 규칙을 뜻한다.

(1) **Softmax 근사 관점.** Hebbian 규칙은 커널 선형 attention 과동치이며, 여기서 $\phi(\cdot)$ 의 역할은 softmax(정확히는 지수 커널) 를근사하는 것이다. 정규화가 있는 지수 커널 (즉 softmax)

은 분리 불가능하여 Transformer 의 이차 복잡도를 유발한다. 그러나 지수 feature map 커널은 Taylor 급수로 근사할 수 있다:

$$\exp(q_i^\top k_j) \approx 1 + q_i^\top k_j + \frac{(q_i^\top k_j)^2}{2!} + \frac{(q_i^\top k_j)^3}{3!} + \dots \quad (4)$$

우리의 다항식 feature map 은 이를 더 일반화한다:

$$\exp(q_i^\top k_j) \approx \phi_p(q)\phi(k)^\top = a_0 + a_1 q_i k_j^\top + a_2 (q_i^\top k_j)^2 + a_3 (q_i^\top k_j)^3 + \dots + a_p (q_i^\top k_j)^p, \quad (5)$$

여기서 학습 가능한 계수 $a_i \in \mathbb{R}$ 는 $a_i = \frac{1}{i!}$ 로 초기화된다. 이 식은 다항식 커널이 학습 가능한 계수를 갖는 **절단된 (truncated) softmax 커널** 근사기임을 뜻한다. 이 직관은 deep memory 와 더 복잡한 attentional bias 로도 일반화된다.

(2) 입력 게이팅 (Input Gating) 관점. 계수 a_i 는 입력 feature 게이팅으로 볼 수 있다. $a_i \rightarrow 0$ 은 차수 i 의 feature map $[x_j]_{|j|=i}$ 를 배제, $a_i \rightarrow 1$ 은 해당 feature 를 유지함을 뜻한다. 이는 RNN 의 게이팅과 비슷하되 메모리가 아닌 **입력에 적용**된다. 모델이 $i \neq 1$ 인 모든 $a_i \rightarrow 0$, $a_1 \rightarrow 1$ 을 학습하면 단순한 $\phi(x) = x$ 가 복원되므로 표현력이 더 크다.

3.2 문맥 기억을 갖는 Long-term Memory (Long-term Memory with Context Memorization)

대부분의 기존 recurrent 모델의 결정적 약점은 이전 메모리 상태는 유지한채 현재 입력에 대해서만 내부 목적함수를 최적화하는 online 한 성질이다:

$$\min_M \ell(M; k_t, v_t) + \text{Ret}_t(M, M_{t-1}), \quad (6)$$

여기서 $\text{Ret}(\cdot, \cdot)$ 은 retention gate 다. 이 식은 새토큰의 손실과 이전 상태 유지항을 함께 최소화함을 뜻한다. 이 online 한성질은 최적화를 단순·빠르게 하지만, 메모리가 개별 토큰을탐욕적으로 기억하여 문맥을 준최적으로만 기억하게 만든다. 더 일반적으로는 각 시점에 전체 문맥에 대해 메모리를 최적화할 수 있다:

$$\min_M \sum_{i=1}^t \ell(M; k_i, v_i). \quad (7)$$

이 식은 지금까지의 모든 토큰을 동시에 잘 저장하도록 메모리를 맞춤을 뜻한다. 그러나 이 엄격한 전역 정식화는 두 한계가 있다: (1) **효율성** — 모든 과거 토큰에 대해 최적화하면 매 갱신마다 추가 제약이 생겨 극단적으로 긴 시퀀스에서 비효율적이고, test time 에 과거 key-value 를 캐싱해야 하여 메모리 소비가 늘어난다. (2) **문맥 가지치기 (pruning)** — 큰 문맥 과제에서 모든 과거 토큰으로 최적화하면 시퀀스 중간의 문맥 변화나 무관한 문맥 때문에 준최적이 될 수 있다.

이를 해결하기 위해, 과거 토큰의 **창 (window)** 에 대해 attentional bias 를 최적화하는 sliding window recurrent 모델을 제시한다. 메모리 모듈 $M(\cdot)$ 과 창 길이 $c \geq 1$ 에 대해:

$$\min_M \sum_{i=t-c+1}^t \gamma_i^{(t)} \ell(M; k_i, v_i), \quad (8)$$

여기서 $\gamma_i^{(t)}$ 는 최적화에서 i 번째 토큰의 영향에 대한 감쇠항이다. 이를 바탕으로, 인기 있는 Delta 학습 규칙보다 엄밀하게 더 강력한 **Omega rule** 을 제시한다:

Omega Rule. key $k_i \in \mathbb{R}^{d_k}$, value $v_i \in \mathbb{R}^{d_v}$, 메모리 모듈 $M(\cdot)$, 국소 문맥 길이 $c \in \mathbb{N}_{\geq 1}$ 가 주어질 때, Omega 학습 규칙에 의한 메모리 갱신은 다음 손실을 gradient descent 로 최적화하는 것으로 정의된다:

$$\min_M \sum_{i=t-c+1}^t \gamma_i^{(t)} \|M(k_i) - v_i\|_2^2. \quad (9)$$

이 갱신 규칙은 $\ell_2(\cdot)$ 을 $\ell_q(\cdot)$ 로 바꿔 q -Omega rule 로도 확장된다. 극단적 경우로 (1) $c = 1$ 이면 갱신이 online(Delta rule) 이 되고, (2) $c = \infty$ (또는 문맥 길이) 이면 모든 과거 토큰에 대한 전역 최적화가 된다. 파라미터 $\gamma_i^{(t)} \in [0, 1]$ 은 과거 토큰에 대한 **하드 (직접) 게이트**로 작동한다. $\gamma_i^{(t)} \rightarrow 0$ 이면 i 번째 토큰의 최적화를 직접 가지치기하고, $\gamma_i^{(t)} \rightarrow 1$ 이면 완전히 포함시킨다. 우리는 $\gamma_i^{(t)}$ 에 입력 의존적 파라미터를 써서 **in-context 가지치기** 능력을 제공한다. sliding window recurrence 설계 덕분에 토큰당 상수 개수 ($\{\gamma_i^{(t)}\}_{i=1}^c$) 의 게이트만 필요해 이런 유연성이 가능하다. 전역 최적화 (식 7) 에 입력 의존적 게이트를 쓰면 파라미터 · 메모리가 크게 늘어 recurrent 모델의 이점이 사라진다.

OmegaNet. 이제 Omega rule 로 메모리를 갱신하는 새 시퀀스 모델 OmegaNet 을 제시한다. 용량을 높이기 위해 key 와 query 에 다항식 커널을 쓴다. 식 9 를 최적화하면 OmegaNet 의 갱신 규칙은:

$$M_t = \alpha_t M_{t-1} - \underbrace{\nabla \sum_{i=t-c+1}^t \gamma_i^{(t)} \|M(\phi(k_i)) - v_i\|_2^2}_{\text{문맥의 surprise}}, \quad (10)$$

또는 선형 메모리 특수한 경우에:

$$M_t = \left(\text{diag}(\alpha_t) - \sum_{i=t-c+1}^t \gamma_i^{(t)} \phi(k_i) \phi(k_i)^\top \right) M_{t-1} - \sum_{i=t-c+1}^t \gamma_i^{(t)} v_i \phi(k_i)^\top. \quad (11)$$

이 식은 (gated) delta rule 의 rank-1 전이 ($I - \beta_t k_t k_t^\top$) 를 **rank- c 로 일반화**한 것을 뜻한다. 메모리 관점에서 Omega rule(OmegaNet) 은 토큰의 surprise 가 아니라, 문맥 내 개별 토큰들의 문맥 인지적 조합에 기반한 **국소 문맥의 surprise** 를 측정한다.

Gradient Descent 를 넘어서. Omega rule 과 “문맥의 test time memorization” 은 식 9 를 임의의 optimizer 로 최적화하도록 확장된다. 두 극단을 예로 든다. 첫째, $c = 1$, $\gamma_i^{(t)} = 1$, gradient descent with momentum 을 쓰면:

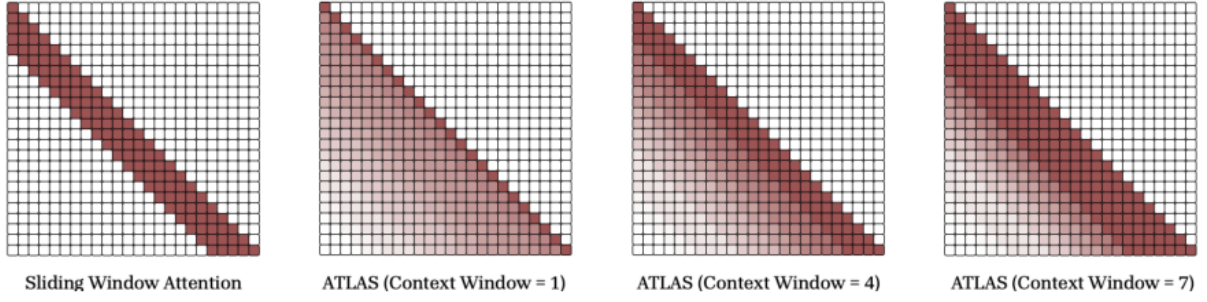


그림 2: SWA 와, 서로 다른 문맥 길이를 갖는 Atlas 또는 OmegaNet 에서의 토큰 의존성 도해. 출처: 원저자, arXiv:2505.23735 Fig.2

$$M_t = \alpha_t M_{t-1} + S_t, \quad (12)$$

$$S_t = \theta_t S_{t-1} - \eta_t \nabla \ell(M_{t-1}; k_t, v_t). \quad (13)$$

이 갱신 규칙은 Titans 의 long-term neural memory 와 동치다. 둘째, 선형 메모리 M , $\gamma_i^{(t)} = 1$, c 를 문맥 길이로 두면 (정규화된) 최소제곱 문제를 최적화하는 것과 동치가 된다:

$$M_t = \min_M \sum_{i=1}^t \|M k_i - v_i\|_2^2. \quad (14)$$

Von Oswald et al. (2023) 는 이를 직접 최적화하고 최적해의 역행렬 항을 Sherman-Morrison 공식으로 재귀적으로 계산할 것을 제안했다 (Mesa-layer). 그러나 이 직접 해는 병렬화 불가능한 학습 비용을 수반하고 선형 행렬형메모리에만 제한되며, 하드 게이팅항 ($\gamma_i^{(t)}$) 이 없어 문맥을가지치기 하지 못해 긴 시퀀스에서 성능을 해친다.

3.3 Omega Rule 의 병렬화 (Parallelizing Omega Rule)

Omega rule 이 Hebbian·Delta 규칙보다 일반적이고 표현력있지만, 대규모 모델 적용은 학습 효율에 달려 있다. online 버전 ($c = 1$) 대비 큰 오버헤드 없는 빠른 병렬 학습 알고리즘을 논한다. 소박한 구현은 위치마다 c 개의 gradient $\nabla \ell \in \mathbb{R}^{d_{in} \times d_{in}}$ 을 실체화해야 하여 d_{in} 이 클 때 메모리·I/O 비용이 크다. 또 TPU·GPU 를 충분히 활용하려면 연산을 텐서화하고 matmul 을 극대화해야 한다. 이에 큰 메모리 오버헤드 없이 병렬 학습을 지원하는 **sliding window masking 전략**을 제안한다.

길이 L 시퀀스를 크기 $b \geq 1$ 의 청크 $S_i = \{x_{(i-1)b+1}, \dots, x_{ib}\}$ 로 나눈다. 각 청크에서 gradient 는 **이전 청크의 마지막 상태에 대해** 계산한다. 명료함을 위해 $\gamma_i^{(t)} = \eta_t$ 로 두면, 청크 크기 $b = 1$ 일 때 갱신규칙은:

$$M_t = \alpha_t M_{t-1} - \eta_t \sum_{i=t-c+1}^t \nabla \ell(M_{t-1}; k_i, v_i). \quad (15)$$

실제로는 완전 recurrent와 완전 병렬 사이 균형을 위해 시퀀스를 청크로나뉘, 청크 내부 (intra-chunk) 는 병렬, 청크 간 (inter-chunk) 은 recurrent 로계산한다. $t' = t - \text{mod}(t, b)$ 로 두면, $t' \leq t < t' + b$ 인시점에 대해 청크 내 갱신 규칙은:

$$M_t = \alpha_t \dots \alpha_{t'} M_{t'} - \underbrace{\sum_{n=t'}^t \alpha_n \dots \alpha_{t'} \eta_n \sum_{i=n-c+1}^n \nabla \ell(M_{t'}; k_i, v_i)}_{G_t}. \quad (16)$$

G_t 는 Titans 와 같은 방식의 gradient 계산에, broadcasting(예: einsum) 시 **sliding window mask** M_s 를 추가로 적용해 얻는다. $c = 1$ 이면 M_s 는 항등 행렬로 줄어든다. $c > 1$ 이면 M_s 는 각대각 원소 바로 앞 $c - 1$ 개 위치도 1로 설정한 항등 행렬이다. 이로써 청크내 모든 gradient 를 실체화하지 않고도 크기 c 의 창에서 gradient 기여를 합할 수 있다.

4 DeepTransformers: Deep Memory 를 갖는 Transformer (DeepTransformers: Transformers with Deep Memory)

최근 연구들은 Transformer 를 associative memory 관점에서 폭넓게 논해왔다. 그렇다면 우리의 메모리 용량 및 Omega rule 논의가 Transformer 에어떤 영향을 주는지 묻는 것은 자연스럽다. 이 절에서는 Omega rule 이 Transformer 및 그 sliding window 대응물 (SWA) 과 어떻게 연결되는지 논하고, Transformer 를 각각 엄밀하게 일반화하는 두 확장을 제시한다.

4.1 메모리의 Online 및 국소 문맥 최적화 (Online and Local Context Optimization of Memory)

Sliding Window Attention 과의 연결. Softmax attention 블록은 Nadaraya-Watson 추정량을 이용한 $\ell_2(\cdot)$ 회귀의 비모수 (non-parametric) 해로 재정식화된다:

$$M^* = \arg \min_M \sum_{i=1}^L s(k_i, q) \|v_i - M\|_2^2 = \sum_{i=1}^L \frac{s(k_i, q)}{\sum_{j=1}^L s(k_j, q)} v_i, \quad (17)$$

여기서 L 은 시퀀스 길이이다. 이 식은 attention 이 각 query 에 대해 key 유사도로 가중된 value 평균으로 메모리를 전역 최적화함을 뜻한다. 최적화를 과거 c 개 토큰으로 제한하면:

$$M^* = \arg \min_M \sum_{i=t-c+1}^t s(k_i, q_i) \|v_i - M\|_2^2 = \sum_{i=t-c+1}^t \frac{s(k_i, q)}{\sum_{j=t-c+1}^t s(k_j, q)} v_i, \quad (18)$$

이는 **sliding window attention (SWA)** 와 동치다. 이 연결은 중요한 통찰을 준다: attention 은 비모수 해일 뿐 아니라 내부 목적함수 (attentional bias) 를 **전역적으로** 최적화하는 반면, 최근 modern recurrent 모델들은 대부분 online learner 다. 우리의 sliding window RNN 과 Omega rule 은 과거 토큰 창에 대해 모수적으로 메모리를 최적화하여 이 간극을 메우며, 개별 토큰이 아닌 문맥을 효과적으로 기억한다.

참고 (각주): 두 예외는 Titans 와 Mesa-layer 다. Mesa-layer 는 모든 과거토큰에 대해 최적화하지만 느린 학습 비용을 치르고, Titans 는 모든 과거토큰에 대해 최적화하되 각 과거 토큰에 대한 암묵적 감쇠항 (모멘텀의 결과) 을 두어 병렬화를 유지한다.

Deep Linear Attention (DLA). 새 baseline 으로, (gated) 선형 attention 의 행렬형 메모리를 deep 신경망 (k 층 MLP) 으로 대체한 Deep (Gated) Linear Attention 을 제시한다. 내적 유사도를 attentional bias 로 쓰고 gradient descent 로 최적화한다:

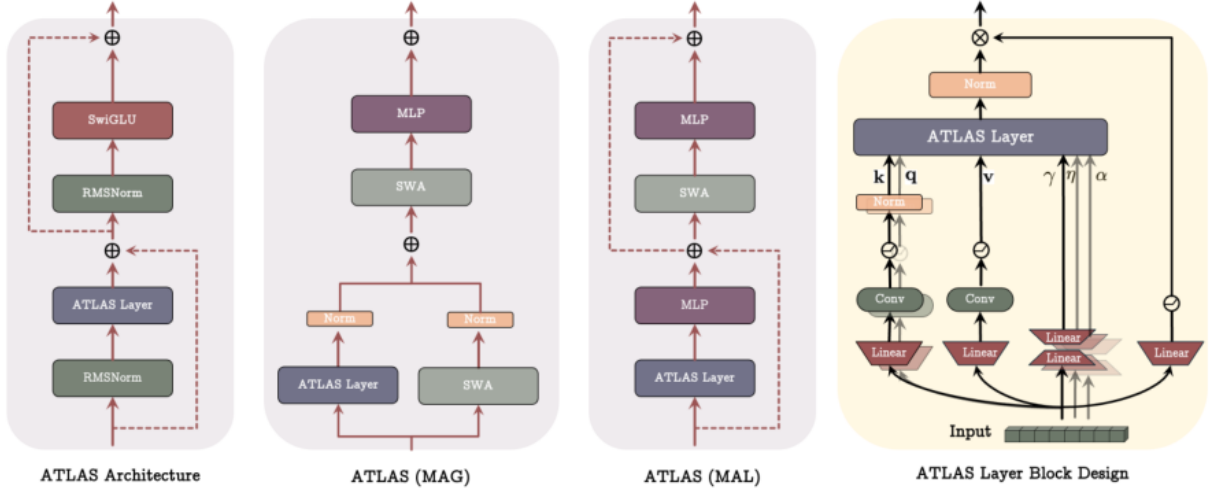


그림 3: Atlas(및 다른 변형들)의 구조와, SWA를 결합한 하이브리드대응물의 도해. 출처: 원저자, arXiv:2505.23735 Fig.3

$$M_t = \alpha_t M_{t-1} - \eta_t \nabla \ell(M_{t-1}; \phi(k_t), v_t), \quad (19)$$

여기서 $\ell(M_{t-1}; \phi(k_t), v_t) = \langle M_{t-1}(\phi(k_t)), v_t \rangle$ 이고 $\phi(\cdot)$ 은 다항식 커널이다. DLA는 선형/비선형 청크학습의 혼합으로 손쉽게 병렬화된다.

Sliding Window Linear Attention (SWLA). SWA와의 연결을 바탕으로, 내적 유사도를 attentional bias로 쓰고 sliding window 손실을 gradient descent로 최적화하는 SWLA블록을 제시한다. 선형 메모리로 닫힌형태를 유도하면:

$$M_t = \alpha_t M_{t-1} - \eta_t \nabla \sum_{i=t-c+1}^t \ell(M_{t-1}; \phi(k_i), v_i) = M_{t-1} + \sum_{i=t-c+1}^t \gamma_i^{(t)} v_i \phi(k_i)^\top. \quad (20)$$

online($c = 1$) 이고 $\phi(\cdot) = (\cdot)$ 이면 이는 선형 attention과 같다.

4.2 메모리 용량과 지수 커널 (Memory Capacity and Exponential Kernels)

먼저 Transformer의 softmax attention(식 1)을 다시 쓴다:

$$y_i = \frac{1}{\sum_{\ell=1}^i \exp(q_i^\top k_\ell / \sqrt{d_{in}})} \sum_{j=1}^i \exp(q_i^\top k_j / \sqrt{d_{in}}) v_j. \quad (21)$$

이 $\exp(\cdot)$ 커널은 분리 불가능하여 recurrence로 쓸 수 없다. 그러나 $\exp(\cdot)$ 커널은 입력을 **무한 차원**으로 보내는 feature map으로 볼 수 있다. 즉 다음을 정의한다:

$$\phi^*(x) = \left(1, x, \frac{x^{\otimes 2}}{\sqrt{2!}}, \frac{x^{\otimes 3}}{\sqrt{3!}}, \dots \right)^\top, \quad \phi_p(x) = x^{\otimes p}, \quad (22)$$

여기서 $x^{\otimes p} = x \otimes x^{\otimes (p-1)}$ 는 Kronecker 곱을 이용한 “self-tensoring” 연산자이고, 따라서:

$$\exp(q_t^\top k_t) = \phi^*(q_t)^\top \phi^*(k_t). \quad (23)$$

이 식은 지수 커널이 무한 차원 공간으로의 분리 가능한 커널임을 뜻한다. 이 커널로 attention(식 21)을 재정식화하면 ($1/\sqrt{d_{in}}$ 은 단순화를 위해 생략):

$$y_i = \frac{1}{\sum_{\ell=1}^i \exp(q_i^\top k_\ell)} \left(\sum_{j=1}^i v_j \phi^*(k_j)^\top \right) \phi^*(q_i) = M_i \phi^*(q_i). \quad (24)$$

이 정식화는 또 다른 중요한 통찰을 준다: **associative memory**로서의 **softmax attention**은 **무한 (unbounded) 메모리**를 가지므로 더 큰 문맥을 파라미터에 더 잘 기억할 수 있다. 이 통찰을 바탕으로, DLA(식 19)에서 다항식 커널을 $\phi^*(\cdot)$ 커널로 대체하여 **DeepTransformers**를 제시한다 (정규화 없는 형태):

$$M_t = M_{t-1} - \nabla \langle M_{t-1}(\phi^*(k_t)), v_t \rangle. \quad (25)$$

선형 메모리 특수한 경우 닫힌 형태는:

$$M_t = M_{t-1} + v_t \phi^*(k_t)^\top = \sum_{i=1}^t v_i \phi^*(k_i)^\top \Rightarrow y_t = M_t \phi^*(q_t) = \sum_{i=1}^t v_i \exp(q_i^\top k_i), \quad (26)$$

이는 **정규화 없는 Transformer**의 출력과 정확히 일치한다. 따라서 DeepTransformers는 softmax attention을 갖는 Transformer의 엄밀한 일반화다.

4.3 Deep Omega Transformer (Dot): Omega 학습 규칙을 갖는 Transformer

위 DeepTransformers는 원래 Transformer처럼 Hebbian 규칙에 기반한다. 그러나 더 강력한 메모리 관리·학습 규칙을 쓰면 성능을 더 높일 수 있다. Hebbian 규칙을 Omega 학습 규칙으로 대체하여, 정규화 없는 **Deep Omega Transformers (Dot)**를 얻는다:

$$M_t = M_{t-1} - \nabla \sum_{i=t-c+1}^t \gamma_i^{(t)} \|M(\phi^*(k_i)) - v_i\|_2^2. \quad (27)$$

선형 메모리. 이 경우 정규화 없는 정식화는:

$$M_t = \left(I - \sum_{i=t-c+1}^t \gamma_i^{(t)} \phi^*(k_i) \phi^*(k_i)^\top \right) M_{t-1} - \sum_{i=t-c+1}^t \gamma_i^{(t)} v_i \phi^*(k_i)^\top, \quad (28)$$

$$y_t = M_t \phi^*(q_t) = \left(I - \sum_{i=t-c+1}^t \gamma_i^{(t)} \phi^*(k_i) \phi^*(k_i)^\top \right) M_{t-1} \phi^*(q_t) - \sum_{i=t-c+1}^t \gamma_i^{(t)} v_i \exp(q_t^\top k_i). \quad (29)$$

$c = 1$ 인 **online** 경우.

$$M_t = \left(I - \eta_t \phi^*(k_t) \phi^*(k_t)^\top \right) M_{t-1} - \eta_t v_t \phi^*(k_t)^\top, \quad (30)$$

$$y_t = M_t \phi^*(q_t) = \left(I - \eta_t \phi^*(k_t) \exp(q_t^\top k_t) \right) M_{t-1} - \eta_t v_t \exp(q_t^\top k_t). \quad (31)$$

이 (정규화 없는) 정식화는 “**Delta rule**을 갖는 **Transformer**”의 일반화로 볼 수 있다. 무한 메모리 덕분에 Dot은 새 key·value를 (원래 Transformer처럼) 덧붙일 뿐 아니라, 해당 key에 대해 이전 상태에서 예측한 value로 새 value를 대체 (오차 정정)한다.

5 Atlas: 높은 용량을 갖는 Locally Optimal 메모리 (Atlas: A Locally Optimal Memory with High Capacity)

Omega rule 로 토큰이 아닌 문맥을 기억하고 다항식 (또는 지수) feature mapping 으로 용량을 높였더라도, 메모리 관리 (즉 key-value 매핑의 최적화) 는 여전히 단순한 gradient descent 에 갇혀 있다. 이 optimizer 는 모델을 낮은품질의 국소 최적점으로 이끌어 긴 문맥에서 성능을 해칠 수 있다. 이를 극복하기 위해, 2차 정보를 근사하면서 대부분 행렬 곱을 사용하고 시퀀스방향으로 병렬화 가능한 **Muon optimizer**(가중치 감쇠 포함) 를 쓸것을 제안한다. Muon 으로 식 9 의 내부 목적함수를 최적화하면:

$$M_t = \alpha_t M_{t-1} - \eta_t \text{NewtonSchulz-}k(S_t), \quad (32)$$

$$S_t = \theta_t S_{t-1} + \nabla \sum_{i=t-c+1}^t \gamma_i^{(t)} \|M(\phi^*(k_i)) - v_i\|_2^2, \quad (33)$$

여기서 c 는 국소 문맥 길이, k 는 Newton-Schulz 연산의 반복 횟수다. Muon 문헌에 따르면 $k \rightarrow \infty$ 일 때 $\text{NewtonSchulz-}k(S_t)$ 는 모멘텀 항 S_t 에 가장 가까운 **준직교 (semi-orthogonal)** 행렬로 수렴하여 2차 정보를 더 낮은오차로 근사한다. 따라서 흥미롭게도 파라미터 k 는 Atlas 의 내부 **test-time compute 파라미터**로 볼 수 있으며, 더 많은 스텝을 쓰면 잠재적으로 더 나은 기억이 가능하다.

5.1 병렬 학습 (Parallel Training)

Atlas 의 학습이 어떻게 병렬화되는지 논한다. 명료함을 위해 $c = 1$ 로 가정한다 (임의의 c 로의 일반화는 § 3.3 을 따른다). § 3.3 처럼 시퀀스를 청크로 나누고 gradient 를 이전 청크의 마지막 상태에 대해 계산한다. 모멘텀은 있으나 Newton-Schulz 는 아직 없는 Atlas 의 recurrence 는:

$$M_t = \alpha_t M_{t-1} + S_t, \quad (34)$$

$$S_t = \theta_t S_{t-1} - \eta_t \nabla \ell(M_{t'}; k_t, v_t). \quad (35)$$

t' 가 이전 청크의 마지막 상태이므로 모든 gradient 를 미리 계산할 수있어 $u_t = \nabla \ell(M_{t'}; k_t, v_t)$ 로 둔다. 그러면:

$$M_t = \alpha_t M_{t-1} + S_t, \quad S_t = \theta_t S_{t-1} - \eta_t u_t. \quad (36, 37)$$

둘째 recurrence 를 전개하면:

$$S_t = \theta_t \dots \theta_1 S_0 - \sum_{i=1}^t \frac{\theta_t \dots \theta_1}{\theta_i \dots \theta_1} \eta_i u_i = \beta_t S_0 - \Theta \odot E \odot G, \quad (38, 39)$$

여기서 $\beta_t = \theta_t \dots \theta_1$, G 는 gradient 행렬, E 와 Θ 는 각각 $\eta \cdot \theta$ 값의 대각행렬, $\odot$ 은 broadcasting 이다. 이 청크별 recurrence 의 핵심 이점은 **모멘텀의 recurrence 가 메모리 상태와 독립**이라는 점이다. 즉 청크시작에서 모든 모멘텀 항을 병렬로 계산할 수 있다. Muon 의 경우 이 모멘텀항들에 Newton-Schulz 를 적용한다:

$$S'_t \leftarrow \text{Newton-Schulz5}(S_t), \quad M_t = M_{t-1} + S'_t. \quad (40, 41)$$

모든 S_t 를 병렬로 계산할 수 있으므로 Newton-Schulz5($\cdot$) 도 병렬로 계산할 수 있다. 이로써 Atlas 는 (근사적) 2 차 내부 optimizer 를 갖는 최초의 병렬화 가능한 recurrent 구조가 된다.

구조 백본 (Architectural Backbone). 최근 modern recurrent 모델들을 따라, key-value-query 를 선형 층으로 projection 한 뒤 크기 4 의 짧은 convolution 층을 둔다. 학습 안정화를 위해 key 와 query 에 정규화를 적용한다. Titans 를 따라 두 하이브리드 변형 MAL·MAG 도 쓴다 (그림 3). Deep memory 구조에서는 residual connection 을 갖는 2 층 MLP 를 쓴다:

$$M(\cdot) = (\cdot) + W_1 \sigma(W_2(\cdot)). \quad (42)$$

이 메모리 구조를 **gated MLP** 층으로 더 확장하면:

$$M(\cdot) = (\cdot) + W_1 (\sigma(W_2(\cdot)) \otimes W_3(\cdot)), \quad (43)$$

여기서 W_1, W_2, W_3 은 학습 가능한 선형 행렬이다. 이 메모리 구조를 갖는 Atlas 를 **Atlas++** 라 부른다.

6 실험 (Experiments)

Atlas, OmegaNet, DeepTransformers, Dot 을 언어 모델링, 상식 추론, needle-in-haystack, in-context recall 과제에서 평가하며 다음 질문에 답한다:

1. Deep memory 는 softmax attention 에 효과적인가? (표 2: Transformer++ vs DeepTransformers)
2. Omega 는 softmax attention 성능을 개선하는가? (표 2: Transformer++, DeepTransformers, Dot)
3. Omega rule 은 더 표현력 있는 메모리 갱신을 주는가? (표 2, 표 6: OmegaNet, Atlas)
4. Locally optimal 메모리 갱신은 효과적인가? (표 2, 표 6: OmegaNet vs Atlas)
5. 비선형 feature mapping 은 효과적인가? (표 6)
6. 제안한 개선이 in-context recall 에서 Transformer 와의 격차를 좁힐 수 있는가? (표 5)
7. 내부 optimizer 가 메모리에 주는 효과는? (그림 6)

설정 (Setup). FineWeb 데이터셋으로 학습 문맥 창 4K 에서 학습한다. 모델 크기는 340M, 400M, 790M, 1.3B 이며 각각 15B, 15B, 30B, 100B 토큰으로 학습한다. baseline 결과는 Behrouz et al. 과 Yang, Kautz, et al. (2024) 에서 가져온다. Perplexity 는 held-out 검증 데이터에서 측정한다. Downstream 과제로 Wikitext, LMB(LAMBADA), PIQA, HellaSwag, WinoGrande, ARC-e, ARC-c, SIQA, BoolQ 를 쓴다.

6.1 언어 모델링과 상식 추론 (Language Modeling and Common-Sense Reasoning)

비하이브리드 모델 중 Atlas 와 OmegaNet 이 perplexity 와 정확도 모두에서 최고 성능을 달성한다. 이는 개별 토큰이 아닌 문맥을 기억하는 능력에 기인한다. 같은 순간적 목적함수 (ℓ_2 손실) 를 쓰되 문맥 창이 1 인 Titans 와 비교하면 non-online 학습 규칙의 효과가 드러난다. 우리 모델은 attention 없이도 하이브리드 모델을 능가하며, MAG 하이브리드 변형은 성능을 더 높인다. 일반화된 Transformer(DeepTransformers, Dot) 는 Transformer++ 대비 일관된 개선을 보인다.

표 2: 언어 모델링 · 상식 추론 성능 (하이브리드 모델은 * 표시).

모델 (760M / 30B)	Wiki ppl ↓	LMB ppl ↓	LMB acc	PIQA	Hella.	Wino.	ARC- e	ARC- c	SIQA	BoolQ	Avg ↑
Trans- former++	25.21	27.64	35.8	66.9	42.2	51.9	60.4	32.5	39.5	60.4	48.69
Deep- Trans- form- ers	20.32	20.67	36.9	68.4	49.8	52.8	65.7	34.9	40.2	61.8	51.31
Dot	19.96	20.15	39.0	69.1	50.7	53.1	66.2	37.0	40.3	63.7	52.39
Ret- Net	26.08	24.45	34.5	67.2	41.6	52.1	63.2	32.8	38.4	57.9	48.46
DeltaNet	24.37	24.60	37.1	66.9	42.0	50.7	64.9	31.4	39.9	59.0	48.97
TTT	24.17	23.51	34.7	67.3	43.9	51.0	64.5	33.8	40.2	59.6	47.32
Gated DeltaNet	21.18	22.09	35.5	68.0	44.9	50.7	66.9	33.1	39.2	59.1	49.69
Samba*	20.63	22.71	39.7	69.2	47.4	52.0	66.9	33.2	39.0	61.2	51.08
Gated DeltaNet- H2*	19.88	20.83	39.2	69.0	48.2	52.6	67.0	35.5	39.4	61.1	51.49
Ti- tans (LMM)	20.04	21.96	37.4	69.3	48.5	52.3	66.3	35.8	40.1	62.8	51.56
Mem- ora	22.28	22.31	38.2	67.8	49.3	53.3	63.6	36.1	40.9	63.0	51.52
SWDT (ours)	19.89	21.52	36.2	68.3	45.2	53.0	65.4	34.2	39.5	59.5	50.1
DLA (ours)	23.12	22.09	36.1	68.0	47.9	52.7	65.8	34.6	39.1	59.6	50.46
OmegaNet (ours)	19.16	20.14	38.7	69.8	50.0	53.3	67.8	36.8	39.6	64.4	52.56
Atlas (ours)	18.92	21.01	39.1	69.7	50.2	53.5	67.5	37.1	40.7	64.3	52.77
At- las++ (ours)	19.04	20.03	39.7	69.7	51.1	53.2	68.2	37.4	40.9	64.4	53.09
Atlas (MAG)	18.62	21.18	40.0	70.3	50.5	53.0	68.1	36.5	41.2	65.0	53.08
Atlas (MAL)	19.07	21.46	38.8	69.2	50.5	53.6	67.3	36.1	41.0	64.5	52.63

표 2 (계속): 1.3B / 100B 토큰.

모델 (1.3B / 100B)	Wiki ppl ↓	LMB ppl ↓	LMB acc	PIQA	Hella.	Avg ↑
Trans- former++	18.53	18.32	42.6	70.0	50.2	52.25
Deep- Trans- formers	15.67	12.63	49.4	72.6	57.0	56.19
Dot	15.28	11.96	50.1	73.3	57.5	57.35
RetNet	19.08	17.27	40.5	70.1	49.2	52.02
Mamba2	16.56	12.56	45.7	71.9	55.7	54.89
DeltaNet	17.71	16.88	42.5	70.7	50.9	52.14

모델 (1.3B / 100B)	Wiki ppl ↓	LMB ppl ↓	LMB acc	PIQA	Hella.	Avg ↑
Gated	16.42	12.17	46.6	72.2	55.8	55.32
DeltaNet						
Samba*	16.13	13.29	44.9	70.9	53.4	54.00
Gated	15.91	12.55	48.8	72.2	56.9	56.18
DeltaNet-						
H2*						
Titans	15.60	11.41	49.1	73.1	56.3	56.82
(LMM)						
Memora	15.90	12.04	48.7	73.1	56.0	55.87
OmegaNet	14.91	11.26	49.7	73.4	57.6	57.23
(ours)						
Atlas	14.97	10.98	50.1	73.9	57.3	57.62
(ours)						
Atlas++	14.40	10.72	50.8	73.5	59.4	58.03
(ours)						

6.2 Long Context: Needle In a Haystack

RULER 벤치마크의 needle-in-haystack 과제 (S-NIAH) 로 유효 문맥 길이와 메모리 용량을 평가한다. Atlas 는 recurrent baseline 대비 매우 좋은 성능을 보여 Titans·DeltaNet 등을 능가한다. 하이브리드 변형은 유효문맥 길이를 더 늘려 학습 문맥 크기의 $\times 4$ 까지 효과적으로 외삽한다. Transformer 류 구조 (DeepTransformers, Dot) 는 긴 문맥에서 하이브리드 Atlas 보다도 뛰어나, 지수 feature mapping 의 중요성을 보인다.

표 3: RULER 의 S-NIAH 성능 (4K 에서 학습, 2K – 16K 평가).

모델	PK-2K	PK-4K	PK-8K	PK-16K	N-2K	N-4K	N-8K	N-16K	W-2K	W-4K	W-8K
TTT	98.4	98.8	98.0	88.4	60.2	36.6	10.2	4.4	78.8	28.0	4.4
DeltaNet	96.8	98.8	98.6	71.4	47.2	15.4	12.8	5.4	46.2	20.0	1.6
Ti-	99.8	98.4	98.2	96.2	100.0	99.8	93.4	80.2	90.4	89.4	85.8
tans											
(LMM)											
Atlas	100	99.2	98.0	97.0	100.0	100.0	93.0	84.0	93.2	90.6	86.2
Samba	98.8	98.0	97.4	97.2	98.8	98.6	96.2	95.6	96.8	90.0	84.0
Gated	99.2	97.8	97.4	98.4	98.0	97.8	96.2	95.8	97.4	96.8	88.4
DeltaNet-											
H2*											
Atlas	100	100	99.4	98.6	100	99.2	97.4	97.0	99.4	98.2	92.4
(MAG)											
Atlas	99.8	99.6	98.4	96.8	99.8	98.0	97.2	96.8	98.0	98.4	92.6
(MAL)											
Deep-	100	100	98.2	97.8	100	98.8	97.8	94.0	95.8	92.2	88.4
Trans-											
form-											
ers											
Dot	100	100	99.6	98.6	100	100	97.8	96.8	99.0	98.4	93.2

6.3 Long Context: BABILong 벤치마크

초대형 시퀀스에서 Atlas 와 Titans 를 비교하기 위해 BABILong 벤치마크로 평가한다. Titans 를 따라 MAC 구조를 쓰되 지속 (persistent) 메모리 토큰 없이 사용하고, 벤치마크 원래 설정대로 fine-tune 한다. 결과 (그림 4) 에서 Atlas 는 1M 문맥 길이까지 Titans 와 대등하지만, 10M 에서

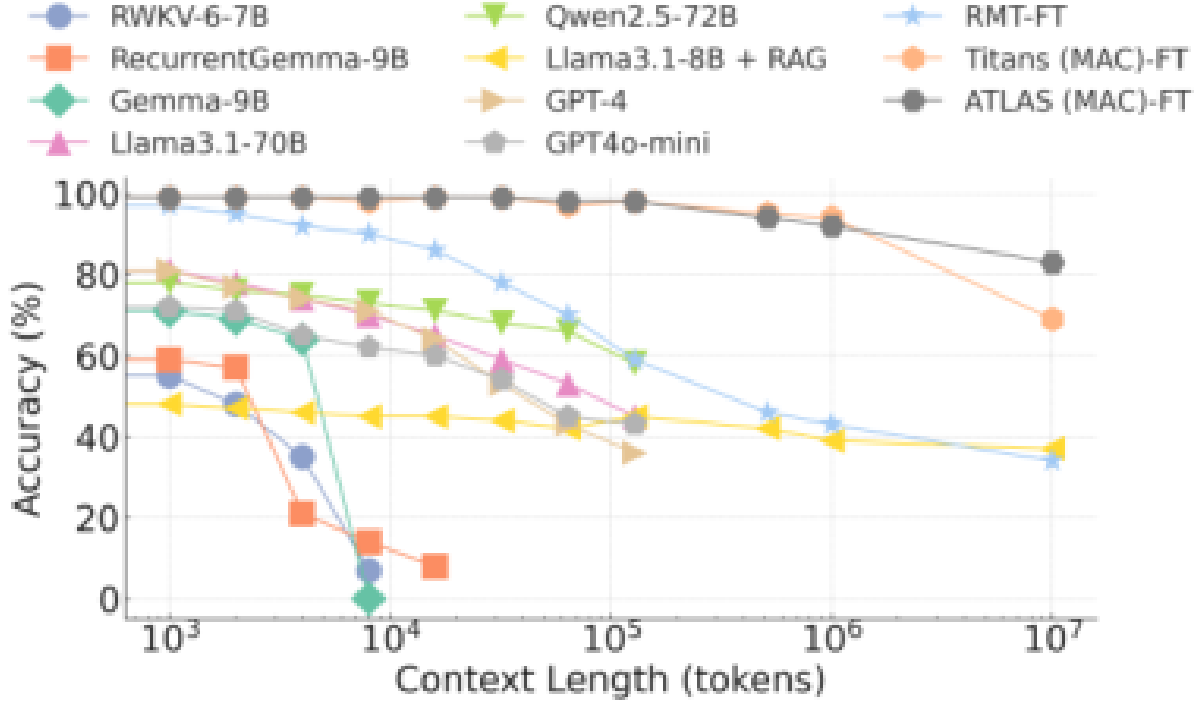


그림 4: BABILong 벤치마크에서 Atlas와 baseline들의 성능. Atlas는 Titans 성능을 능가하며 10M 문맥 길이까지 효과적으로 확장한다. 출처: 원저자, arXiv:2505.23735 Fig.4

Titans는 성능이 떨어지는 반면 Atlas는 성능을 유지하며 10M 문맥에서 +80% 정확도를 달성한다. 이는 (1) Muon을 통한 메모리 관리, (2) 다항식 커널을 통한 더 나은 메모리 용량, (3) 개별 토큰이 아닌 문맥을 기억하는 성질 덕분이다.

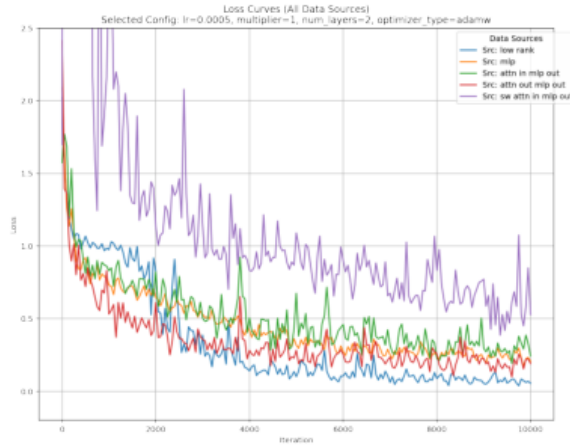
이후로는 제시한 개선의 중요성을 보이기 위해 recurrent 구조 (Atlas, OmegaNet)에 집중한다.

6.4 학습 가능성 실험 (Learnability Experiments)

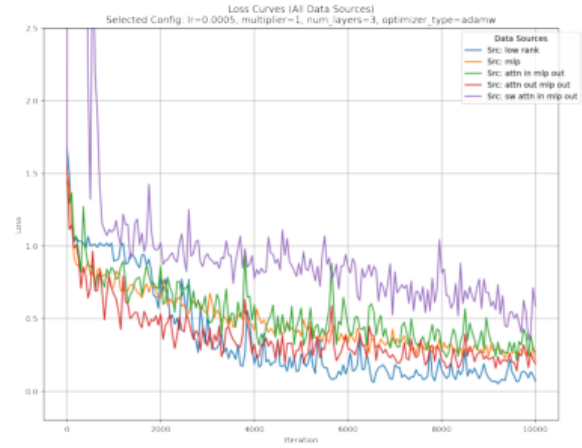
작은 MLP가 online 방식으로 함수를 학습하는 능력을 분석했다. 튜플 시퀀스 $(i_1, o_1), \dots, (i_t, o_t) (i_j, o_j \in \mathbb{R}^d)$ 에 대해, MLP M 을 online으로 학습하여 손실 $\text{loss}_j = \|i_j - o_j\|_2^2 / \|o_j\|_2^2$ 를 최소화한다. 시각 j 에서 gradient $\nabla_{M, \text{params}} \text{loss}_j$ 를 계산하고 Adam, RMSprop, SGD로 파라미터를 갱신한다. 다섯 가지 시퀀스-투-시퀀스 함수를 연구한다:

1. **Low Rank Mappings:** 무작위 저랭크 행렬 $W = XY (X \in \mathbb{R}^{d \times k}, Y \in \mathbb{R}^{k \times d})$ 로 $o_j = W^\top i_j$.
2. **MLP Mappings:** 1 입력 · 1 은닉 · 1 출력 MLP M (GELU, 확장 없음)로 $o_j = M(i_j)$.
3. **Attention+MLP Mapping:** causal masked attention을 입력에 적용해 o'_j 를 얻고 $o_j = M(o'_j)$.
4. **Attention Outputs as Inputs:** 위와 같되 o'_j 를 입력, o_j 를 출력 시퀀스로.
5. **Sliding Window Attention + MLP Mapping:** 설정 3과 같되 full attention 대신 창 512의 sliding window attention 사용.

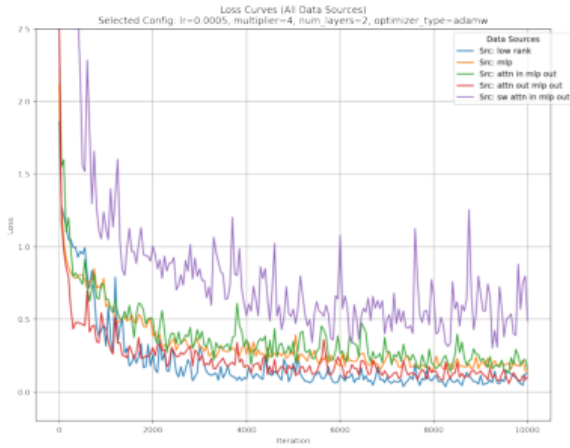
$d = 256$ 으로 두고 다섯 설정의 위치별 손실 곡선을 그림 6에 보인다. 모든 설정에서 모델은 결국 $\text{loss}_j < 1$ 인 비자명한 매핑을 학습한다. 설정 4의 attention이 유발한 입력 상관 덕분에 설정 2보다 빠르게 학습되고, 가장 단순한 설정 1에서 대체로 가장 잘 학습된다. 모델은 이전 입력 · 출력을 (부분적으로) 기억해야 하는 설정 3과 5에서 가장 못한다. 흥미롭게도 용량 요구가 더 클 것으로 예상되는 설정 3이 설정 5보다 오히려 더 잘 학습되는데, 우리는 학습 알고리즘이 오래된 입력을 ‘잊



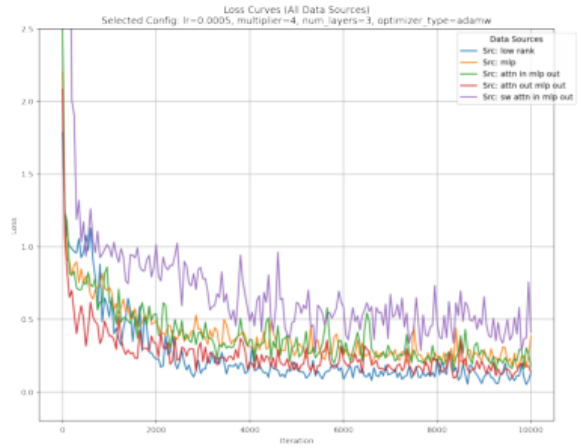
(a) $\mathcal{M}$ with 2 hidden layers and no expansion.



(b) $\mathcal{M}$ with 3 hidden layers and no expansion.



(c) $\mathcal{M}$ with 2 hidden layers and 4x expansion.



(d) $\mathcal{M}$ with 3 hidden layers and 4x expansion.

그림 6: 다양한 하이퍼파라미터 설정에 대한 손실 곡선. 출처: 원저자, arXiv:2505.23735 Fig.6

지 (forget)’ 못하기 때문에 sliding window 설정 (5) 에서 손실이 더나빠진다고 가정한다. 다만 attention 이 무작위 초기화 벡터에서 계산되어 attention 행렬이 (학습된 LLM 처럼) 뻔족하지 않다는 점은 분석의 한계다.

6.5 추가 실험: In-context Recall, MAD 합성 벤치마크, Associative Recall

먼저 recall· 기억· 압축· 복사 과제를 평가하는 합성 벤치마크 MAD 로 평가한다 (표 4). Atlas 는 모든 측면, 특히 기억 (memorization) 에서 최고 성능을 달성하여 용량 강화 요소의 중요성을 보인다.

표 4: MAD 합성 벤치마크 성능. Atlas 는 Transformer 를 포함한 모든 baseline 을 능가한다.

모델	Compression	(Noisy) ICR	Fuzzy ICR	Selective Copying	Memorization	Average
Trans-formers	49.4	100	48.2	95.9	83.8	75.46
Gated DeltaNet	44.8	100	32.5	96.2	81.7	71.04
Titans	49.6	100	49.7	99.4	83.5	76.44
OmegaNet (ours)	50.9	100	54.2	99.6	90.2	78.98

모델	Compression	(Noisy) ICR	Fuzzy ICR	Selective Copying	Memorization	Average
Atlas (ours)	51.6	100	54.9	99.6	91.4	79.50

In-context recall 은 recurrent 신경망에 가장 어려운 벤치마크 중 하나다. SWDE, NQ, DROP, FDA, SQUAD, TQA 로 평가한다 (표 5). Transformer 가 여전히 최고이지만, Atlas 와 OmegaNet 은 경쟁력 있는 성능으로 state-of-the-art recurrent 모델을 능가한다. 이는 더 나은 메모리 관리 · 용량에 기인한다.

표 5: In-context recall 성능. Transformer 가 최고이나, 문맥 기억과 다항식 feature map 설계가 격차를 좁힌다.

모델	SWDE	NQ	DROP	FDA	SQUAD	TQA	Average
Transformers	84.9	23.0	28.4	72.5	48.1	64.4	53.55
Gated DeltaNet	63.2	19.1	26.7	33.4	39.6	59.7	40.28
Titans	65.1	20.7	27.2	37.3	42.6	61.0	42.31
OmegaNet (ours)	67.4	21.1	27.2	39.0	43.2	60.9	43.13
Atlas (ours)	66.8	21.9	27.4	40.7	44.1	61.3	43.70

마지막으로 Multi-Query Associative Recall(MQAR) 과제에서 Atlas 와 Dot 을 평가한다 (그림 7). 두 모델 모두 baseline 대비 좋은 성능을 보이며, Atlas 는 DeltaNet 등 state-of-the-art 모델 대비 **메모리 크기당 최고성능**을 달성한다.

6.6 Ablation 연구와 스케일링 패턴 (Ablation Study and Scaling Patterns)

Atlas 구성 요소에 대한 ablation 과 스케일링을 평가한다 (표 6). 결과는 다음을 보인다: (1) gated MLP 같은 더 강력한 메모리 구조가 Atlas 성능을 더 높인다; (2) 하이브리드 변형이 성능을 개선하며 MAG 가 MAL 보다 낫다; (3) 다항식 mapping 과 deep memory 는 문맥 기억 (Omega rule) 을 쓸 때 특히 중요하다. 그림 5 는 국소 문맥 길이 c 의 효과를 보이는데, c 가 커질수록 성능이 좋아진다. 이는 필요할 때 문맥을 가지치기하는 게이팅 파라미터 γ 덕분이다.

표 6: Atlas 의 Ablation 연구 (760M). 모든 구성 요소가 성능에 긍정적으로 기여한다.

설정	언어 모델링 ppl ↓	상식 추론 acc ↑
Atlas	19.97	52.77
+Gated MLP Memory	19.53	53.09
+Attn (MAG)	19.90	53.08
+Attn (MAL)	20.26	52.63
Linear Memory	21.03	49.74
w/o Muon	19.65	52.56
$c = 1$	21.98	49.26
w/o Polynomial Mapping	22.14	50.57

모델 크기. 그림 8 은 파라미터 수에 대한 Atlas·OmegaNet 의 스케일링을 보인다. 두 모델 모두 크기증가에 따라 모든 스케일에서 baseline 대비 더 낮은 perplexity 를 달성하는 좋은 스케일링 패턴을 보인다.

문맥 길이. 그림 8 은 문맥 길이에 대한 스케일링도 보인다. 두 모델 모두 높은 메모리 용량 덕분에 문맥 길이를 늘려도 잘 확장된다.

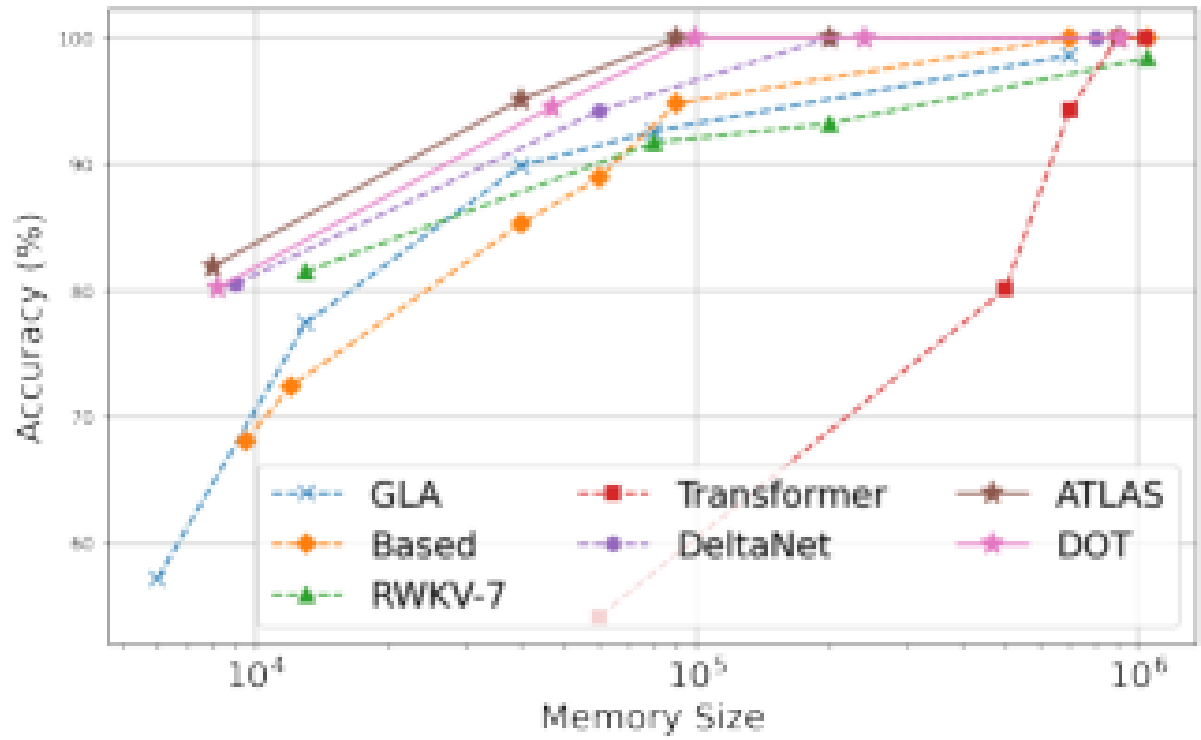


그림 7: Associative memory recall(MQAR) 결과. 출처: 원저자, arXiv:2505.23735 Fig.7

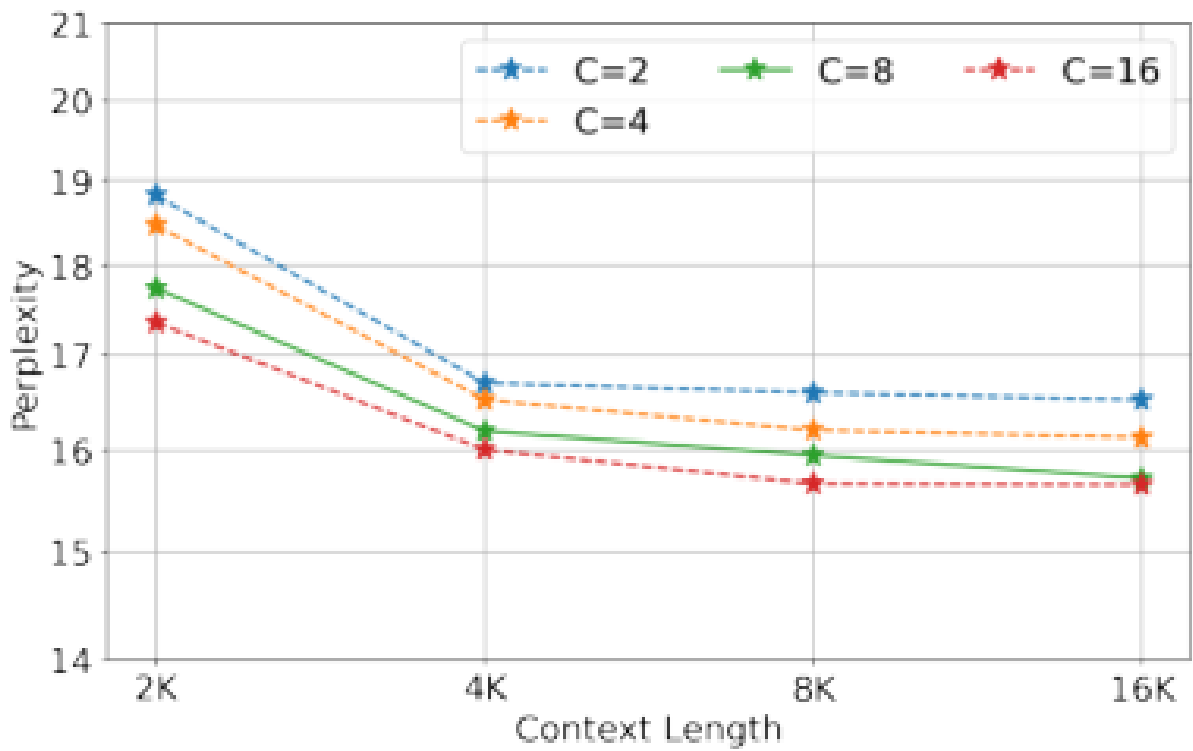


그림 5: OmegaNet 에서 국소 문맥 길이 c 가 서로 다른 전역 문맥길이별 성능에 주는 효과. 출처: 원저자, arXiv:2505.23735 Fig.5

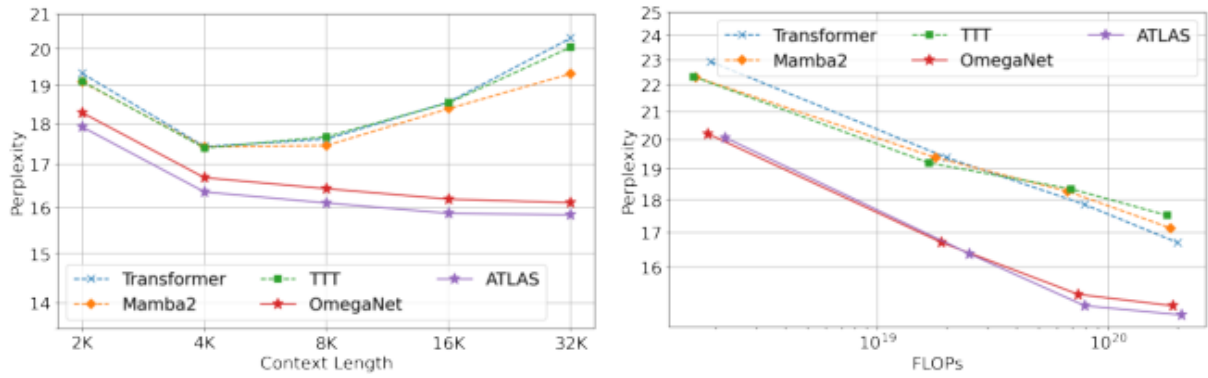


그림 8: (왼쪽) 학습 문맥 길이 및 (오른쪽) FLOPs 에 대한 Atlas·OmegaNet 의 스케일링 패턴. 출처: 원저자, arXiv:2505.23735 Fig.8

7 결론 (Conclusion)

우리는 modern recurrent 모델의 핵심 한계 — 한정된 메모리 용량, online 전용 갱신, 약한 메모리 관리 — 를 다루기 위해 새 long-term memory 모듈 **Atlas** 를 도입했다. 제안한 sliding window 학습 규칙, 고차 feature mapping, 진보한 메모리 optimizer 는 이 도전을 극복하는 원리적이고 확장가능한 접근을 제공한다. 실증적으로 우리 모델들 (OmegaNet, Atlas, DeepTransformers, Dot) 은 다양한 벤치마크에서 Transformer 와 최근 RNN 변형대비 일관된 개선을 이룬다. 이론적으로 우리는 메모리 용량과 최적화동역학에 대한 통찰을 제공하여, 선행 연구에서 관찰된 문맥 길이 한계에대한 설명을 제시했다.

부록 (Appendix)

부록 A. 추가 관련 연구 (Additional Related Work)

Modern 선형 Recurrent Neural Network. Transformer 의 이차복잡도와 long-context 한계를 완화하기 위한 효율적 recurrent 대안들이 개발되었다. 초기 (RetNet, RWKV, S5) 는 데이터 비의존적 전이 행렬과 Hebbian 유사 갱신을 썼고, 이후 세대는 입력 의존적 파라미터를 도입했다 (선형 RNN, RWKV6). 이들은 delta rule 기반의 더 표현력 있는 갱신 규칙도 탐구했다. 나아가 deeper 모델로 확장하며 delta-rule 유사 갱신 (TTT) 이나 forget gating 을 갖는 데이터 의존적 모멘텀 갱신 (Titans) 을 활용했다. 최근 Siems et al. (2025) 은 토큰당 다중 gradient descent 갱신을 적용해 특히 state tracking 에서 더 표현력 있는 모델을 제안했다. 비선형 recurrence 를 갖는 RNN 과 그 빠른 학습에 대한 연구도 있다.

Fast Weight Programs. 선형 층을 key-value associative memory 로보는 개념은 Hopfield network 로 거슬러 올라가며, 이후 fast weight programmer 맥락에서 동적 fast program 을 RNN 에 통합해 쓰기 가능한 메모리로 삼았다. 학습 패러다임으로는 Hebbian 학습과 delta rule 이 가장 두드러진다.

Hopfield Networks. 우리의 정식화는 key-value 매핑 학습이라는 associative memory 의 넓은 개념 위에 세워져 있다. Hopfield(1982) 는 에너지함수 최소화로 key-value 쌍을 저장하는 초기 신경 구조를 도입했다. 전통적 Hopfield network 는 벡터형 메모리 용량 한계로 쓰임이 줄었으나, 여러연구가 지수 커널을 에너지 함수에 도입하는 등으로 용량을 강화했고, 현대화된 Hopfield network 와 Transformer 의 관계도 연구되었다.

부록 B. Miras 프레임워크 (Miras Framework)

Behrouz, Razaviyayn, et al. (2025) 는 associative memory 를 정의 2 로형식화했다 (정의 1 과 동일한 $M^* = \arg \min_M \mathcal{L}(M(K); \mathcal{V})$, 식 44). 이 목적함수를 반복적 알고리즘으로 최적화하면 메모리 갱신 규칙이 나오고, 시퀀스 모델은 inner loop(메모리 파라미터 최적화) 와 outer loop(나머지 파라미터 최적화) 의 두 수준을 갖는 메타 in-context learner 다.

각주: 여기서 “선형” 은 빠른 학습 · 추론 절차를 가리키며 recurrence 공식을 뜻하지 않는다. Titans, Yaad, Moneta, Memora, TTT 등은 비선형 recurrence 기반이지만 학습 · 추론이 빠르다.

B.1 예시. 선형 attention 은 내적 유사도 $\tilde{\ell}_t := \langle M_{t-1}k_t, v_t \rangle$ 의 gradient descent 최적화로 정의된다:

$$M_t = M_{t-1} - \eta_t \nabla \langle M_{t-1}k_t, v_t \rangle = M_{t-1} + \eta_t v_t k_t^\top. \quad (45, 46)$$

회귀 손실을 쓰면 DeltaNet 을 얻는다:

$$M_t = M_{t-1} - \eta_t \nabla \|M_t k_t - v_t\|_2^2 = \left(I - \eta_t k_t k_t^\top \right) M_{t-1} + v_t k_t^\top. \quad (47)$$

부록 C. 뒷받침 증명 (Supporting Proofs)

명제 1 증명. $K = [k_1 \cdots k_m] \in \mathbb{R}^{d_k \times m}$, $V = [v_1 \cdots v_m] \in \mathbb{R}^{d_v \times m}$ 로 두면 문제는 Frobenius norm $\|MK - V\|_2^2$ 최소화가 되고, 정확한 기억은 $MK = V$ 를 요구한다. 벡터화하면 $(K^\top \otimes I_{d_v}) \text{vec}(M) = \text{vec}(V)$ 로 md_v 개 방정식, $d_k d_v$ 개 미지수다. key 가 선형 독립이면 $\text{rank}(K) = m$ 이므로 시스템 행렬은 완전 행 랭크 md_v 를 갖고, 가해성은 $md_v \leq d_k d_v$, 즉 $m \leq d_k$ 를 요구한다. $m \leq d_k$ 이고 K 가 완전 열 랭크이면 Moore-Penrose 유사역행렬로 정확한 보간해 $M^* = VK^\top$ 를 구성할 수 있어 상한이 tight 하다. 또한 스텝 $0 < \eta < 2/\lambda_{\max}(KK^\top)$ 의 full-batch gradient descent 는 최소 노름 보간해 $M^\dagger = VK^\top$ 로 수렴한다 (gradient descent 의 암묵적 편향). 같은 랭크 제약이 선형 · multi-head attention 의 “low-rank bottleneck” 을 지배한다. $\square$

정리 1 증명 (스케치). ReLU MLP 는 조각별 아핀이다. 고정된 활성화패턴에서 $M(\cdot) = A \cdot + B$, $A = W^{(L_M)} D^{(L_M-1)} W^{(L_M-1)} \cdots D^{(1)} W^{(1)}$ (각 $D^{(\ell)}$ 은 활성화 유닛을 고르는 $\{0, 1\}$ 대각 행렬) 이다. 모든 key 가 같은 선형 영역에 있으면 (작은 섭동 후 일반적으로 성립) M 은 단일아핀 변환이다. 정확한 기억은 $AK = \mathcal{V}$ 를 뜻하므로 $m = \text{rank}(K) \leq \min\{\text{rank}(A), d_k\}$ 이다. A 의 랭크 는 각 경로의 최소 폭과 즉각 입력 차원의 곱으로 제한되어 $\text{rank}(A) \leq \sum_{i=1}^{L_M} \min_{j \geq i} \{d_h^{(j)}\} d_h^{(i)} = O(d_k d_v \sum_{i=1}^{L_M} \min_{j \geq i} \{d_h^{(j)}\} d_h^{(i+1)})$ 이다. $\square$

명제 2 증명. 차수 정확히 ℓ 인 d_k 변수 단항식의 수는 stars-and-bars 로 $\binom{d_k + \ell - 1}{\ell}$ 이고, $\ell = 0$ 부터 p 까지 합하면 hockey-stick 항등식으로 $D = \binom{d_k + p}{p}$ 이다. $\Phi = [\phi_p(k_1) \cdots \phi_p(k_m)] \in \mathbb{R}^{D \times m}$ 로 두면 정확한 기억은 $M\Phi = V$ 를 요구하는데, 랭크의 sub-multiplicativity 로 $\text{rank}(M\Phi) \leq \text{rank}(\Phi) \leq D$ 이다. target 이 선형독립이면 ($V = [e_1, \dots, e_m]$) $\text{rank}(V) = m$ 이고, $m > D$ 이면 시스템이 풀리지 않는다. 따라서 어떤 optimizer 도 $D = \binom{d_k + p}{p}$ 개를 초과해 기억할 수 없고, 고정 p 에 대해 $\binom{d_k + p}{p} = \Theta(d_k^p)$ 이므로 용량은 $O(d_k^p)$ 로 상한이 정해진다. $\square$

부록 D. 모든 구조의 상세 정식화 (Detailed Formulations)

- **D.1 DLA:** 내적 유사도 $\min_M \langle M(k_t), v_t \rangle$ 를 gradient descent 로 최적화, $M_t = \alpha_t M_{t-1} - \eta_t \nabla \ell(M_{t-1}; k_t, v_t)$; 선형메모리에서 $W_t = \alpha_t W_{t-1} + v_t k_t^\top$ (gated 선형 attention).
- **D.2 SWLA:** sliding window 목적함수 $\min_M \sum_{i=t-c+1}^t \langle M_{t-1}(k_i), v_i \rangle$; 선형메모리에서 $M_t = \alpha_t M_{t-1} - \sum_{i=t-c+1}^t \eta_i^{(t)} v_i k_i^\top$.

- **D.3 OmegaNet:** 내적 대신 ℓ_2 회귀를 써서 $M_t = \alpha_t M_{t-1} - \sum_{i=t-c+1}^t \eta_i^{(t)} \nabla \|M_{t-1}(\phi(k_i)) - v_i\|_2^2$ (ϕ 는 다항식 feature map).
- **D.4 Atlas:** OmegaNet 과 같은 내부 목적함수를 Muon(가중치 감쇠포함) 으로 최적화: $M_t = \alpha_t M_{t-1} + \text{Newton-schulz5}(S_t)$, $S_t = \theta_t S_{t-1} - \sum_{i=t-c+1}^t \eta_i^{(t)} \nabla \|M_{t-1}(\phi(k_i)) - v_i\|_2^2$.

부록 E. 실험 세부 (Experimental Details)

Wikitext, LMB, PIQA, HellaSwag, WinoGrande, ARC-e/c, SIQA, BoolQ 를 쓴다. baseline 은 Behrouz et al. 에서 가져온다. T5 tokenizer(어휘 32K) 를 쓰고 학습 길이 4K(SWA 는 2K) 를 쓴다. AdamW(학습률 4e-4, cosine annealing, 배치 0.5M 토큰, 가중치 감쇠 0.1) 로 학습한다.

표 7: 구조 세부.

모델	Block	Dim	Head	Peak LR	Token
170M	12	768	16	3e-3	15B
340M	24	1024	16	1.5e-3	15B
760M	24	1536	16	1.25e-3	30B
1.3B	18	2048	8	7e-4	100B

메모리 구조는 특별한 언급이 없으면 확장 인자 4, GELU 활성화, residual connection, 그리고 각 체크 끝의 layer norm 을 갖는 2 층 MLP $M(x) = x + W_1 \sigma(W_2 x)$ 를 쓴다.

부록 F. 추가 실험 결과 (Additional Experimental Results)

F.1 언어 모델링 · 상식 추론 (소규모). 모든 스케일결과를 표 8 에 보인다. Atlas 와 OmegaNet 은 modern 선형 recurrent 모델과 Transformer 대비 더 낮은 perplexity 와 더 나은 정확도를 달성한다. 완전 recurrent 버전조차 Samba, Gated DeltaNet-H2 같은 하이브리드를 능가한다. MAG·MAL 하이브리드는 성능을 더 높여, recurrent long-term memory 와 attention 의 상보적 역할을 보인다. SWLA 가 GLA·RetNet 보다 나은 점은 개별 토큰이 아닌 문맥을 기억하는것의 중요성을 시사한다.

표 8 (일부): 340M / 15B 토큰.

모델	Wiki ppl ↓	LMB ppl ↓	Avg ↑
Transformer++	31.52	41.08	42.92
RetNet	32.50	49.73	42.54
GLA	28.51	43.02	44.09
Mamba	30.83	40.21	43.59
DeltaNet	28.65	47.30	44.04
TTT	27.44	34.19	44.51
Gated DeltaNet	27.01	30.94	45.42
Moneta	26.19	29.31	46.44
Yaad	26.61	29.11	45.93
Memora	27.16	30.44	45.51
DLA (ours)	27.93	35.09	44.76
SWDT (ours)	26.98	33.95	45.31
OmegaNet (ours)	26.03	28.76	46.93
Atlas (ours)	25.88	28.54	47.28